```
# Step 1 – Load the Dataset
# Importing Libarires
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Reading files
df = pd.read_csv("/content/sales_data_sample.csv", encoding="Latin-1")
```

```
# Step 2 – Perform Basic Data Understanding
# Check Data
df.head()
```

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDERDATE	STATUS	QTR_ID	MONTH_ID	Y
10	10223	37	100.00	1	3965.66	2004-02-20	Shipped	1	2	
21	10361	20	72.55	13	1451.00	2004-12-17	Shipped	4	12	
40	10270	21	100.00	9	4905.39	2004-07-19	Shipped	3	7	
47	10347	30	100.00	1	3944.70	2004-11-29	Shipped	4	11	
51	10391	24	100.00	4	2416.56	2005-03-09	Shipped	1	3	

5 rows × 26 columns

```
Start coding or generate with AI.
```

```
# Check structure
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2823 entries, 0 to 2822
Data columns (total 25 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   ORDERNUMBER       2823 non-null   int64
 1   QUANTITYORDERED   2823 non-null   int64
 2   PRICEEACH         2823 non-null   float64
 3   ORDERLINENUMBER   2823 non-null   int64
 4   SALES             2823 non-null   float64
 5   ORDERDATE         2823 non-null   object
 6   STATUS            2823 non-null   object
 7   QTR_ID            2823 non-null   int64
 8   MONTH_ID          2823 non-null   int64
 9   YEAR_ID           2823 non-null   int64
 10  PRODUCTLINE       2823 non-null   object
 11  MSRP              2823 non-null   int64
 12  PRODUCTCODE       2823 non-null   object
 13  CUSTOMERNAME      2823 non-null   object
 14  PHONE             2823 non-null   object
 15  ADDRESSLINE1      2823 non-null   object
 16  ADDRESSLINE2      302 non-null    object
 17  CITY              2823 non-null   object
 18  STATE             1337 non-null   object
 19  POSTALCODE        2747 non-null   object
 20  COUNTRY           2823 non-null   object
 21  TERRITORY         1749 non-null   object
 22  CONTACTLASTNAME   2823 non-null   object
 23  CONTACTFIRSTNAME  2823 non-null   object
 24  DEALSIZE          2823 non-null   object
dtypes: float64(2), int64(7), object(16)
memory usage: 551.5+ KB
```

```
# Summary statistics
df.describe()
```

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	QTR_ID	MONTH_ID	
count	2823.000000	2823.000000	2823.000000	2823.000000	2823.000000	2823.000000	2823.000000	282
mean	10258.725115	35.092809	83.658544	6.466171	3553.889072	2.717676	7.092455	200
std	92.085478	9.741443	20.174277	4.225841	1841.865106	1.203878	3.656633	
min	10100.000000	6.000000	26.880000	1.000000	482.130000	1.000000	1.000000	200
25%	10180.000000	27.000000	68.860000	3.000000	2203.430000	2.000000	4.000000	200
50%	10262.000000	35.000000	95.700000	6.000000	3184.800000	3.000000	8.000000	200
75%	10333.500000	43.000000	100.000000	9.000000	4508.000000	4.000000	11.000000	200
max	10425.000000	97.000000	100.000000	18.000000	14082.800000	4.000000	12.000000	200

```
# Step 3 – Clean the Dataset
# Remove null values
df = df.dropna()
df.head()
```

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDERDATE	STATUS	QTR_ID	MONTH_ID	Y
10	10223	37	100.00	1	3965.66	2/20/2004 0:00	Shipped	1	2	
21	10361	20	72.55	13	1451.00	12/17/2004 0:00	Shipped	4	12	
40	10270	21	100.00	9	4905.39	7/19/2004 0:00	Shipped	3	7	
47	10347	30	100.00	1	3944.70	11/29/2004 0:00	Shipped	4	11	
51	10391	24	100.00	4	2416.56	3/9/2005 0:00	Shipped	1	3	

5 rows × 25 columns

```
# Remove duplicates
df = df.drop_duplicates()
df.head()
```

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDERDATE	STATUS	QTR_ID	MONTH_ID	Y
10	10223	37	100.00	1	3965.66	2/20/2004 0:00	Shipped	1	2	
21	10361	20	72.55	13	1451.00	12/17/2004 0:00	Shipped	4	12	
40	10270	21	100.00	9	4905.39	7/19/2004 0:00	Shipped	3	7	
47	10347	30	100.00	1	3944.70	11/29/2004 0:00	Shipped	4	11	
51	10391	24	100.00	4	2416.56	3/9/2005 0:00	Shipped	1	3	

5 rows × 25 columns

```
# Convert columns to correct datatype
df["ORDERDATE"] = pd.to_datetime(df["ORDERDATE"])
df.head()
```

	QTR_ID	MONTH_ID	YE
	1	2	
	4	12	
	3	7	
	4	11	
	1	3	

```
# Step 4 — Calculate Essential Sales Metrics
# Total Sales
total_sales = df["SALES"].sum()
print("Total Sales:", total_sales)
```

```
Total Sales: 506562.52
```

```
# Average Sales
avg_sales   = df["SALES"].mean()
print("Average Sales:", avg_sales)
```

```
Average Sales: 3446.003537414966
```

```
# Maximum Sale
max_sale    = df["SALES"].max()
print("Maximum Sale:", max_sale)
```

```
Maximum Sale: 9774.03
```

```
# Minimum Sale
min_sale    = df["SALES"].min()
print("Minimum Sale:", min_sale)
```

```
Minimum Sale: 652.35
```

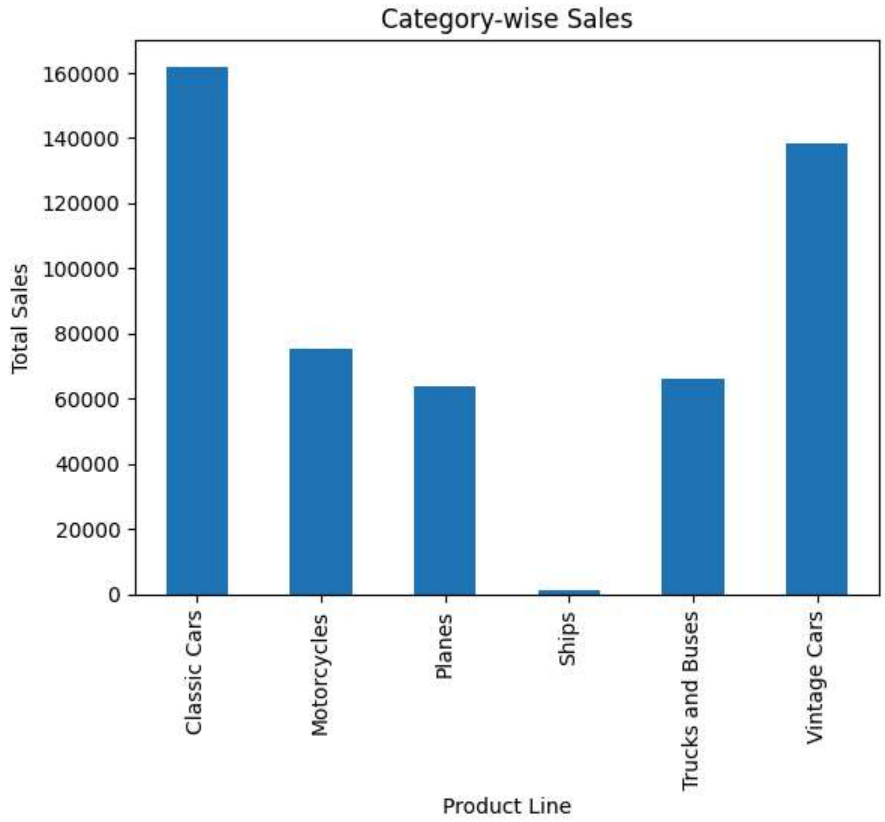
```
# Category-wise totals
# Category-wise:
df.groupby("PRODUCTLINE")["SALES"].sum()
print("Category-wise Totals:")
print(df.groupby("PRODUCTLINE")["SALES"].sum())
```

```
Category-wise Totals:
PRODUCTLINE
Classic Cars        161870.46
Motorcycles          75476.67
Planes               63772.09
Ships                 1089.36
Trucks and Buses     66020.96
Vintage Cars        138332.98
Name: SALES, dtype: float64
```

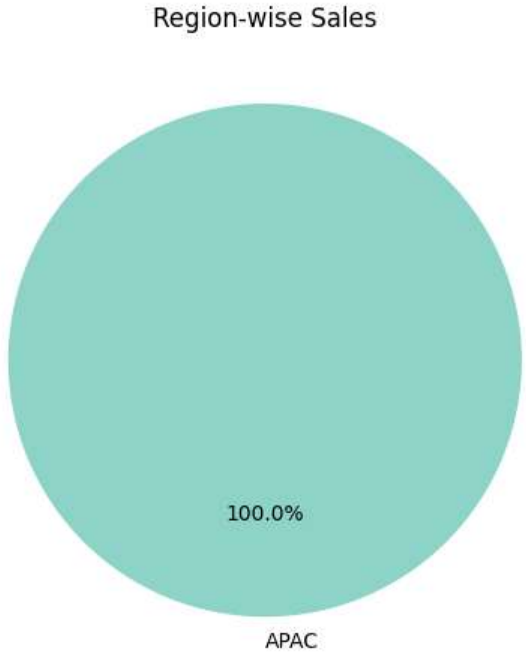
```
# Region-wise
df.groupby("TERRITORY")["SALES"].sum()
print("Region-wise Totals:")
print(df.groupby("TERRITORY")["SALES"].sum())
```

```
Region-wise Totals:
TERRITORY
APAC    506562.52
Name: SALES, dtype: float64
```

```
# Step 5 – Create Visualizations
# Category-wise Sales:
df.groupby("PRODUCTLINE")["SALES"].sum().plot(kind="bar")
plt.title("Category-wise Sales")
plt.xlabel("Product Line")
plt.ylabel("Total Sales")
plt.show()
```

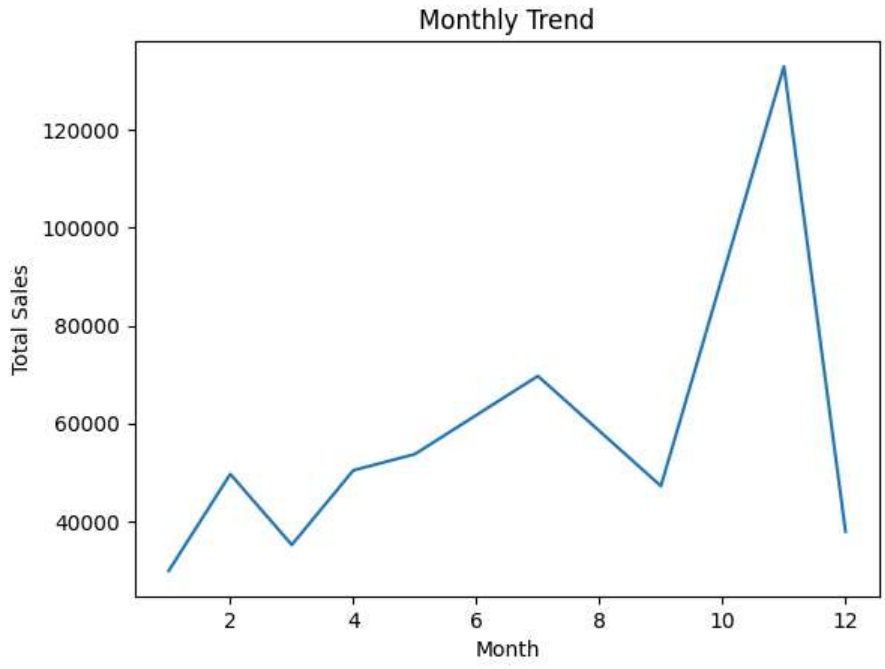



```
# Region-wise Sales
df.groupby("TERRITORY")["SALES"].sum().plot(kind='pie',autopct='%1.1f%%', startangle=90, cmap='Set3')
plt.title("Region-wise Sales")
plt.ylabel("")
plt.tight_layout()
plt.show()
```



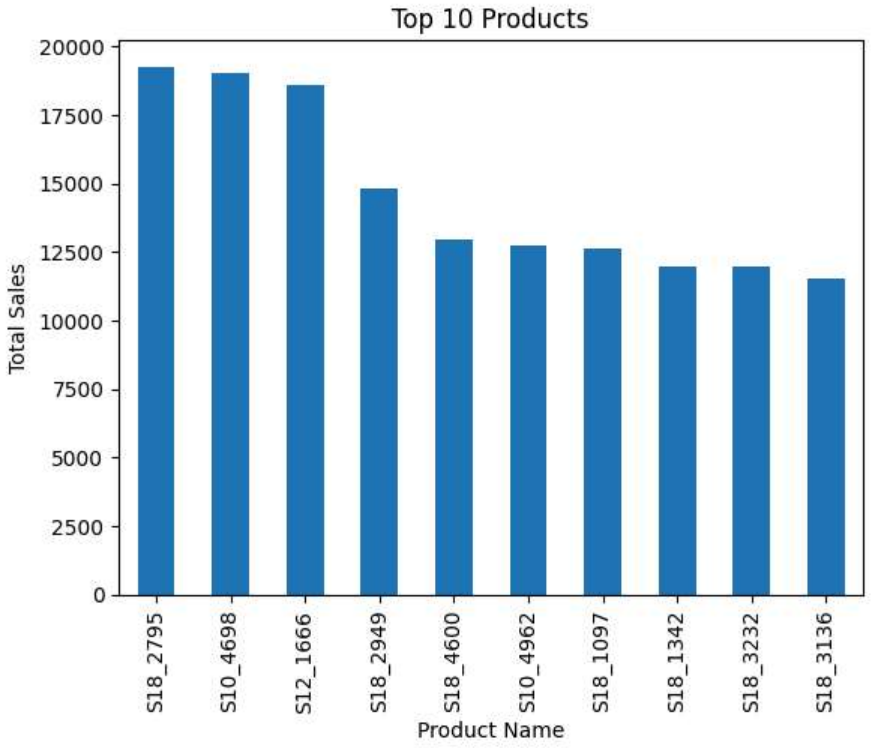

```
# Monthly Trend:
df["MONTH"] = df["ORDERDATE"].dt.month
df.groupby("MONTH")["SALES"].sum().plot(kind="line")
plt.title("Monthly Trend")
plt.xlabel("Month")
plt.ylabel("Total Sales")
```

```
Text(0, 0.5, 'Total Sales')
```



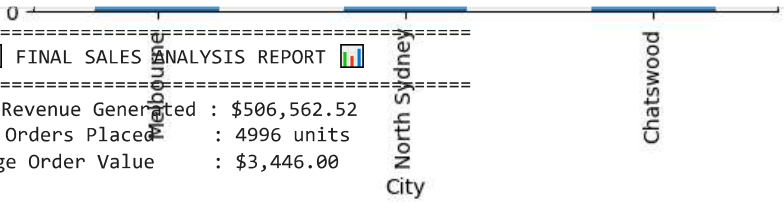

```
# Top 10 Products:
df.groupby("PRODUCTCODE")["SALES"].sum().sort_values(ascending=False).head(10).plot(kind="bar")
plt.title("Top 10 Products")
plt.xlabel("Product Name")
plt.ylabel("Total Sales")
```

```
Text(0, 0.5, 'Total Sales')
```




```
# Top 10 Cities By Revenue
top_cities = df.groupby("CITY")["SALES"].sum().sort_values(ascending=False).head(10)
print("--- Top 10 Cities by Revenue ---")
print(top_cities)
df.groupby("CITY")["SALES"].sum().sort_values(ascending=False).head(10).plot(kind="bar")
plt.title("Top 10 Cities by Revenue")
plt.xlabel("City")
plt.ylabel("Total Sales")
```

```
# --- FINAL PROJECT SUMMARY REPORT ---
print("================================================")
print("       📊 FINAL SALES ANALYSIS REPORT 📊         ")
print("================================================")

# 1. Financials
print(f"💰 Total Revenue Generated : ${df['SALES'].sum():,.2f}")
print(f"📦 Total Orders Placed     : {df['QUANTITYORDERED'].sum()} units")
print(f"🆙 Average Order Value     : ${df['SALES'].mean():,.2f}")

# 2. Top Performers
top_cat = df.groupby("PRODUCTLINE")["SALES"].sum().idxmax()
top_cat_val = df.groupby("PRODUCTLINE")["SALES"].sum().max()
print(f"\n🔥 Top Performing Category : {top_cat} (${top_cat_val:,.2f})")

top_city = df.groupby("CITY")["SALES"].sum().idxmax()
top_city_val = df.groupby("CITY")["SALES"].sum().max()
print(f"📈 Highest Revenue City    : {top_city} (${top_city_val:,.2f})")

top_prod = df.groupby("PRODUCTCODE")["SALES"].sum().idxmax()
print(f"🛍️ Number 1 Product (Code) : {top_prod}")

# 3. Deal Size breakdown
print("\n📦 Deal Size Contribution:")
print(df.groupby("DEALSIZE")["SALES"].sum().sort_values(ascending=False).apply(lambda x: f"${x:,.2f}"))
print("================================================")
```




```
================================================
       📊 FINAL SALES ANALYSIS REPORT 📊
================================================
💰 Total Revenue Generated : $506,562.52
📦 Total Orders Placed     : 4996 units
🆙 Average Order Value     : $3,446.00

🔥 Top Performing Category : Classic Cars ($161,870.46)
```